

Presentations

This is a selection of three sets of presentations at meetings and a bunch of examples for some styles that come with **ConT_EXt**. In **MkIV** some of the older styles have been dropped. There are more presentations, starting mid 1990's: some are lost, others make no sense showing (or have articles instead) and I have no time anyway to collect all of them.

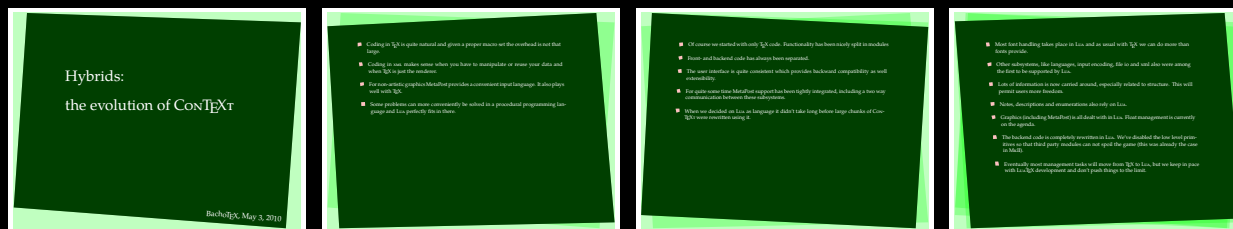
We show the first four pages but you can find the complete files in the **ConT_EXt** distribution. This file is generated automatically so it's also a snapshot. Maybe some day I find the time to add some more.

There are more files in the source tree because it doesn't make sense to provide all **pdf** files. Some files are to be generated with **LuaT_EX** because they discuss something (**MkIV**) that in the meantime has been superseded. Keep in mind that these talk are snapshots and might have presenting the state at that time, some experiments, or just have been teasers.

bachotex-2010-clash



bachotex-2010-move



bachotex-2011-cld-and-mkvi



bachotex-2013-bits

Bits and pieces:

ConTeXt
MetaPost
Lua and more

Hans Hagen
EuroBachOT_εX
May 2013

ConTeXt recently done	ConTeXt next on the todo list	MetaPost recently done
MetaPost next on the todo list	Lua recently done	Lua next on the todo list
LuaTeX recently done	LuaTeX next on the todo list	Fonts next on the todo list
Manuaals	Scripts	Speed

ConTeXt
recently done

- some more cleanup of old left-over
- most mechanisms now use the new kind of parameter abstraction
- only a few fundamental incompatibilities (right of mechanisms, more control)
- slow introduction of hooks and extensions via setups
- first version of new multi-column routines
- replacing mechanism that have a (too) complex implementation
- exploring the mix (e.g. chemistry with Alan)
- remove some ugly left-overs from MetaPost
- normalized dynamic fonts (mostly interfacing)

ConTeXt
next on the todo list

- more definitive split between generic and context (generated)
- some math extensions, maybe already math dictionaries
- normalize all styles and modules
- play a bit more with the Lua parser/parser
- add more integrated bid layout support
- finalize experimental (auto)script code
- investigate what more is needed in the CXX interface
- check multi-lingual interface translations (not entirely in sync now)

bachotex-2013-luatex

LuaTeX for dummies

(so you can still leave)

Hans Hagen
EuroBachOT_εX
May 2013

The TeX perspective	The Lua perspective	The hybrid perspective
The complications	The future	This workshop
Just plain	A bit less plain	Hardly plain

The TeX perspective

- it started out as pdfTeX
- then got merged with AlgaTeX
- but we left out the ugly bits of both
- its exclusively utf-8
- its math machinery got extended with OpenType like features
- there are no fundamental extensions as its impossible to agree in them

The LUA perspective

- it just a Lua engine
- it has some extra libraries on board
- you don't even have to use TeX
- but there are hooks into the TeX machinery
- and we can go further by loading libraries

bachotex-2013-sense

How about those
typographic virtues:
do they still make sense?

Hans Hagen
EuroBachOT_εX
May 2013

Typesetting	In the process	But eventually
Targeting paper	Towards displays	The state of affairs
Take ligatures	Accent battles	Justification
Backslashes	Endangered features	The future of TeX

Typesetting

- somehow we turned sounds into speech into language
- and after that it may have started with writing in the sand
- followed by painting on cave walls
- or maybe carving symbols in wood
- and figuring out some writing system
- that made it possible to chisel thoughts in stone
- and eventually putting blobs on paper
- that we somehow managed to map onto pixels
- but
- typesetting only happened very lately

In the process

- markup optimized the basic shapes (to suit the language)
- and kept inventing new symbols
- more and more automated typesetting and rendering and printing
- and of course this went with all kind of silly arguments
- think of this:
- for ages scribes were the only way to get high end arabic typesetting
- and now we can do this mostly automatic (in good and bad ways)
- so we endangered yet another craft

bachotex-2013-speed

BachoT_EX

Grandpa's toolbox

(making closets)

Dad's hobby

(cleaning closets)

Uncle's friends

(talking closets)

bachoTeX 2016

-

+

- there's closets and closets
- take the ones you put stuff in ... e.g. that you buy at ikea
- you have to assemble them so you get out your toolbox
- this kid sits next to you wondering what that tool is
- what is natural for you to use might not be so for them
- but grandpa likes to carry over his knowledge and experiences

-

closets

+

- but ... kids get their information from the internet, not from you
- (they watch discovery channel or national geographic and know a lot)
- or they look at vloggers (no bloggers) trying to learn something
- and they keep moving on ... and on
- do gp's live long tools really make sense to them?

-

interest

+

- grandpa also has a vlogger, he is called knuth
- gp is as locked into tex as the kids are into apps
- he's a do-it-him-self kind of person
- will his grandkids love watching him crafting
- and hear his stories about meetings and journals

-

grandpas

+

bachotex-2017-emoji

[illegible]

```

For now one has to enable the feature:

\definefontfeature[overlay]{default}{ccmp=yes, colr=yes, dist=yes}
\definefontfeature[svg]{default}{svg=yes}
\definefontfeature[bitmap]{default}{sbix=yes}

\definefontfeature[colored]{default}
{ccmp=yes, dist=yes,
 colr=yes, svg=yes, sbix=yes}

Defining a font is not different from others

\definefont{\FontName}{Font} [sequence]{overlay}
\definefontsynonym{emoj}{sequence}{overlay}

As is using:

{\FontName}{Font}{resolvedemoj}{woman}}
\expandafter

```

bachotex-2017-variablefonts

We thrive in information-thick worlds because of our marvelous and everyday capacity to select

idealize, isolate, discriminate, distinguish, screen, pigeonhole, pick over, sort, integrate, blend
inspect, filter, lump, skip, smooth, chunk, average, approximate, cluster, aggregate, outline, sum
marize, itemize, review, dip into, flip through, browse, glance into, leaf through, skim, refine
enumerate, glean, synopsis, winnow the wheat from the chaff and separate the sheep from the
goats

edit, single out, structure, highlight, group, pair, merge, harmonize, synthesize, focus, organize, condense, reduce, boil down, choose, categorize, catalog, classify, list, abstract, scan, look into, idealize, isolate, discriminate, distinguish, screen, pigeonhole, pick over, sort, integrate, blend, inspect, filter, lump, skip, smooth, chunk, average, approximate, cluster, aggregate, outline, sum-

We thrive in information-thick worlds because of our marvelous and everyday capacity to select

edit, single out, structure, highlight, group, pair, merge, harmonize, synthesize, focus, organize, condense, reduce, boil down, choose, categorize, catalog, classify, list, abstract, scan, lead into, idealize, isolate, discriminate, distinguish, screen, pigeonhole, pick over, sort, integrate, blend, inspect, filter, lamp, skip, smooth, chunk, average, approximate, cluster, aggregate, outline, summarize, itemize, review, dip into, flip through, browse, glance into, leaf through, skim

We thrive in information-thick worlds because of our marvelous and everyday capacity to select, edit, single out, structure, highlight, group, pair, merge, harmonize, synthesize, focus, or

ganize, condense, reduce, boil down, choose, categorize, catalog, classify, list, abstract, scan, look into, idealize, isolate, discriminate, distinguish, screen, pigeonhole, pick over, sort, integrate, blend, inspect, filter, lump, skip, smooth, chunk, average, approximate, cluster, aggregate, outline, summarize, itemize, review, dip into, flip through, browse, glance into, lean through, skim, refine, enumerate, clean, mosey, winnow the wheat from the chaff and

We thrive in information-thick worlds because of our marvelous and everyday capacity to select, edit, single out, structure, highlight, group, pair, merge, harmonize, synthesize, focus, organize, condense, reduce, boil down, choose, categorize, catalog, classify, list, abstract,

scan, look into, idealize, isolate, discriminate, distinguish, screen, pigeonhole, pick over, sort, integrate, blend, inspect, filter, lump, skip, smooth, chunk, average, approximate, cluster, aggregate, outline, summarize, itemize, review, dip into, flip through, browse, glance

we're ready for them

So, in practice, for most $\text{\TeX}$ users it's just a font that has to be supported by $\text{\TeX}$ and friends.

- Condition: no tricks, no fuzzy heuristics, just the specification should be enough.

code with the specification.

bachotex-2018-fonteffects

Modern Latin

BachOTeX 2018 — Hans Hagen

Why oh why

- In ConTeXt we have a mechanism to apply effects to a glyph stream.
- An active user on the ConTeXt mailing list wondered if that could be applied to specific fonts.
- The particular interest concerned the possibility to bolden fonts.
- I don't really like effects and not all fonts are suitable for it.

What are effects

Normal effects are implemented using the 'effects' mechanism, which already dates way back in MetaPost and of course is also available for MetaFont.

```
def\effecteffect
[font]
[selectivevector,
selectivechannel,signs]
effect\selecteffect[font]effect\selecteffect
```

effect effect effect
effect effect effect
effect effect effect

inner outer both

bachotex-2018-mp

MetaPost Extensions A few examples

BachOTeX 2018 — Hans Hagen

History

- We started using MetaPost some two decades ago and immediately used the pdf version.
- We used special colors plus special to communicate extension, for instance each color and shape.
- This mechanism was elegantly improved and extended. Basic mechanism, the basis, needed an extra pass.
- When we moved to LuaTeX and again we started using pre- and postscripts to easily communicate with the public.
- Currently we use a lot of Lua from within able to communicate during the MetaPost run with ConTeXt. This permits closer interaction.

Colors

```
def\setcolor
draw image {
draw image {
fill\setcolor\setcolor fill\setcolor\setcolor
fill\setcolor\setcolor fill\setcolor\setcolor
fill\setcolor\setcolor fill\setcolor\setcolor
} defined {1,0,0}
draw image {
fill\setcolor\setcolor fill\setcolor\setcolor
fill\setcolor\setcolor fill\setcolor\setcolor
fill\setcolor\setcolor fill\setcolor\setcolor
} defined {1,0,0}
} defined\setcolor
\setcolor\setcolor
```

bachotex-2019-followingup

The follow up LuaMetaTeX

BachOTeX May 2019

Hans & Alan

History

- This talk is most about how we can use LuaTeX to extend document applications. We already discussed that.
- This talk is most about how we can use LuaTeX to extend document applications. We already discussed that.
- This talk is most about how much fun it would be to have a MetaPost extension and use what ConTeXt and LuaTeX could do with it. We just can't afford it.
- This talk is most about more complexity, but it is about keeping things simple. It's about turning a burden into a pleasure.
- To quote the Riverside frontman: I hope you all have here a lot stronger than you talk when you come here. This talk is about turning lead into gold.

LuaMetaTeX
From lead to gold

Colors

- We have the weight and experience of about 40 years of TeX and its usage on our shoulders.
- Good old TeX got extended. eTeX, Omega (Alpha), pdfTeX, LaTeX, XeTeX, and LuaTeX (in LuaTeX) showed up.
- The dot output got complemented by pdf.
- Binary fonts were replaced by Type 1 fonts that could be replaced by the container formats OpenType and TrueType. Variable fonts were introduced.
- Many got supported in OpenType math, thanks to MiKTeX.
- Unicode got accepted and still is nowadays the preferred input encoding.
- The community supported the development of many fonts that found their place in distributions.
- Alternative plain TeX like macro packages like TeX and ConTeXt both evolved into large collections of resources.

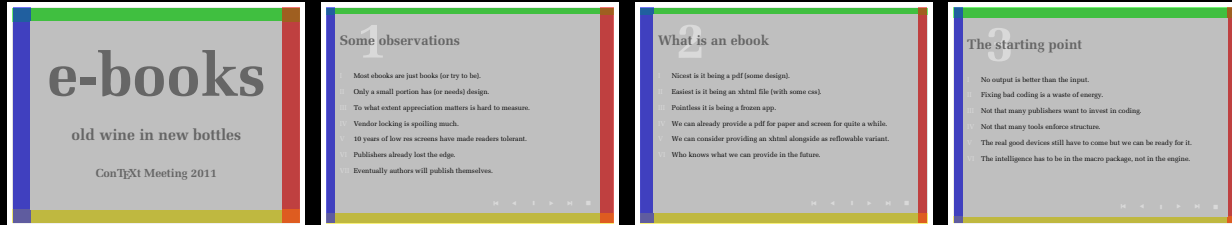
LuaMetaTeX
Where do we stand

Colors

- There is no doubt that TeX is a success. We can find new examples but also some horrible looking documents.
- A large distribution is no guarantee for quality and continuous success, nor is the number of academic research users.
- A ConTeXt user doesn't need that much, just the LaTeX library will do, plus a bunch of MetaPost, compiled by a reasonable set of fonts.
- Currently all that is embedded in a large extension, although we always use only a small, reasonable subset.
- Getting the whole machinery up and running from scratch is never easy and is not trivial.
- The source code base is rather large and compilation is complex. It builds on decades of heavy work for all platforms and facilities all demands.
- What we consider gold could also be seen as lead in disguise. Some day they might be needed to go back to where we came from.

LuaMetaTeX
Are we good?

context-2011-ebook-export



context-2011-mathml-update



context-2011-metapost-how-we-adapt



[illegible][illegible]

Breakers 2012	Existing sources	Breakers 2012	Existing sources	Breakers 2012	Existing sources
What is lexing - Computer languages have mandate structure. - You can avoid errors by checking the input. - Editors can help by coloring reserved words, concept etc. - Consistency in coloring different languages makes sense When did we start - We wrote our first editor begin 80's. - An extension, called Followed when we moved to the XEDIT. - When MINOTA was no longer feasible we moved on to PRED, XEDITOR (quite dumb). - When we were able to use coding like C, we moved to CEDITOR. - I proposed some simplifications for LEX and METAPRED (support for multiple formats etc.).		Breakers 2012 Side effect of MKIV - SCITE got LEX based lexing (external lexing). - Later on, I had already written some lexers for the pretty printers. - In 1996 it got me made some more advanced lexers. - These ship with CONQUEST, LEX, PRED, LEX, CLEX, METAPRED, text. Characteristics - The lex lower supports nested lexing of LEX and METAPRED. - Lexical token (only) checking is provided. - Unfortunately there is no lexing for SCITE on MAXIMA (not that I care to make maximals). - It is a pity that I have no access to SCITE material as with the original LEX interface. - On my part old lexicals keep lying in the somewhere bins (at the end).		Breakers 2012 The future - I will improve the current lexers. - An IDE, lexers will be added at some point. - I might make an LEX/LEX-variant that supports nested LEX.	

context-2012-mixed-columns

Breakdown 2012 The script

Output

- `ly` collects content paragraph wise.
- In between it can trigger the so called output routines.
- At last moment you can do something with the results.
- One of the things you can do is package all collected so far in a page.

Boxes

- In `lualatex` we can also intercept content at some places.
- For instance before and after each paragraph is processed.

Breakdown 2012 The script

Columns

- `ly` has no concept of columns.
- You need to fake them by filling with the width and spilling boxes.
- Often we run our `TeX` code (no output content).
- For some local applications we use simple columns.
- In far instance `TeX` can use a mixed size and multi-column model.
- Columns are another (independent) mechanism, strongly grid based.
- Traditional multi-columns are being replaced by a new mechanism: mixed columns.

Pitfalls

- Fontsize: page, first or last column, each column (aligned, immediate).
- Complex: mixing floats around is more complex than in single columns.
- Nesting: how about columns inside columns.
- Relinking: can be hard taking all into account.

Breakdown 2012 The script

Questions

- Do complex column mechanisms still make sense given the move to electronic paper.
- If so, what functionality should be provided.

context-2012-the-script

Breakdown 2012 The script

Some myths

- `CONTEXT` looks at last line plus `ly` and expects users to program macros.
- `CONTEXT` depends on `TeX`.

The truth

- On the average users don't have to program. ConTeXting is not programming.
- As `ly` lacks conventional handling and job-control, helpers are provided.
- All context users can still program a lot, but not all need that.
- All context users can directly run `CONTEXT`, but why should they?

A few facts

- The `CONTEXT` distribution provides a sort of ecosystem.
- In `TeX` indeed we use `TeX` for some job control.
- But in `TeX` all is left context done in `ly`.
- Two scripts play an important role: `mixcols` and `context`.

Breakdown 2012 The script

The 'mixcols' script

- Locates and runs scripts, has a lot of helpers provided.
- It is in fact the `TeX` runner on top the `ly` script.
- It knows about files and the environment we run in.
- It has some features that makes it easier to integrate in services.
- The way we don't need much (and avoid potential conflicts in some).

The 'context' script

- It runs `CONTEXT` and keeps track of how many runs are needed.
- Contrary to its `TeX` ancestor it is not needed for make testing etc.
- It has a few extensions that are loaded on demand: `extra`.

Breakdown 2012 The script

A regular run

```
context [-:run] filename
```

Running from an editor

```
context --script filename
```

Running from an service

```
strut --path=filepath -script context filename
```

Breakdown 2012 The script

Controlling the rendering

```
--noheaderlist
--nofooterlist
--noindexlist
--noindexlist
--noindexlist
--noindexlist
```

Controlling with `ctx` files

```
--ctxname
```

Also in preamble

```
<?context-directive job style email.ctx >
```

context-2012-visual-debugging

Breakdown 2012 Visual debugging

How it started

- Some 15 years ago I wanted some more feedback.
- First I started out a way to translate boxes, lines, etc.
- Some aspects were tricky, like stretch and shrink (as a `TeX` part), fillers, leaders, etc.
- I gave some presentations and it was not to see the parallel faces.
- As debugging does not work, it is somewhat interfering.
- When not needed there is no overhead but we still disable it at some places.

Do we need it

- I wonder if anyone ever used it.
- Some of the helpers are quite handy, like `ly` and `lualatex`.
- So there had to be provided anyway, so: when to stop?

Breakdown 2012 Visual debugging

All kind of debugging

- We have some debugging, much shows up when writing new code.
- Think of fonts, math, graphics, characters, etc.
- Some make us aware in `TeX`, so they're gone, but some more show up.
- In that time there will all be surrounded (or more: from in modules).

Breakdown 2012 Visual debugging

Why we kept it

- When cleaning up the code I had to decide to keep it or redo it as it could be done `ly`-ish.
- But as we already had some `ly`-based extra it made sense to redo it.
- The old code is still there as module (also because it had some more useful).

How it worked

- In `TeX` procedures are overloaded.
- So effectively when enabled, `TeX` can run become macros.
- We use rules (and leaders) to window properties.
- Some constructs interfere so we need to compensate side effects.

Breakdown 2012 Visual debugging

How it works

- The helper were a rather trivial quick job as we had a lot in place already.
- Interpreting the mode list and existing structures.
- The new colors, rules and text box stuff can be overlaid.
- Control over what gets rendered at the `ly` end.
- Control over what gets shown by using layers.
- A second most time went into mechanism, choices and optimization.
- Some structures interfered with (hard-coded) expectations in the backend.
- When I decided to run layers I had to adapt some other code (mostly out of efficiency).
- There is room for more (but first I want the bits of `ly` 5.2).

context-2012-xml-news

Breakouts 2012	Processing XML, some basics
Topics	
• processing	
• selecting	
• flushing	
• testing	
• basic's only	

```

Processing
\nprocessfile (name) (filename) (setup)
\nprocessbuffer (name) (filename) (setup)
\nloadonly (name) (filename) (setup)

Loading
\nload (name) (filename) (setup)
\nloadbuffer (name) (buffername) (setup)

```

```

Injecting elements

val nil (mode) (pattern)
val first (mode) (pattern)
val last (mode) (pattern)
val from (mode)

val raw (mode) (pattern)
val content (mode) (pattern)
val attr (mode) (pattern)
val tag (mode)
val text (mode) (pattern)

```

```
Bash: $ cd ..
```

```
Processing XML... some.html
```

```
Injecting attributes
```

```
\set      (node) (name)
```

```
\setattr  (node) (name) [default]
```

```
\setAttribute (node) (pattern) (name)
```

```
\setattr!name (node) (pattern) (name) (default)
```

```
Injecting properties
```

```
\setContent (node) (pattern)
```

```
\setName    (node)
```

```
\setNameSpace (node)
```

context-2013-math

Math:

progress or standing still

Hans Hagen
ConT_EXt Meeting
September 2013

Math as script	Alphabets	Heavy bold	Radicals
Primes	Accents	Stackers	Fences
Directions	Structure	Italic correction	Big
Macros	Unscripting	Combining fonts	Tracing

Math as script

- math can be input using the TeX system, MacSiml, calculator like sequences,
- but although part control MacSiml all play down to good old TeX
- but haven't officially a script, Octave treats it as such, but without control

```
B ((x + 1) / a + 1)^2 = (x - 1) / b %  
B \left( \frac{x+1}{a} + 1 \right)^2 = \frac{x-1}{b} %  
%defined operator "==" class = "r"  
%definec ...</macro> <macro>+<macro> <macro>1</macro>  
%definec ...</macro>
```

There is recognition of math as a program (but not standardized) script.

Alphabets

- the shape (style) of a character determines its meaning
- but in most cases an type is embodied in *one* character
- and tagged with some rendering directive, often indicating a font style
- in traditional T₈g we have alphabets in different fonts, so we're talking switches
- in TeXmacs and D₉TeX we have alphabets with standardized code points (but gaps too)
- this has big advantages for communicating, transferring data etc.
- but a math engine still has to deal with *sans* input as well
- multiple axis: types, alphabets, styles, variants, shapes, modifiers

We're off better but the gaps are an annoyance

context-2013-speed

Speed:

why it matters
and why we care

Hans Hagen
ConTeXt Meeting
September 2013

Speed	Pages per minute
What happens	What we can do

Speed

- speed matters in a edit-run-preview cycle although this is mostly perception
- the more the interface, the slower it goes, but you seldom set something up so that it is not much of a burden
- everything you provide gets used at some point, also in sufficient ways, so best know your weak spots
- lots of local (grouped) tweaks leads to many mechanisms lacking in unseen, grouping matters
- writing use of functionality can have drastic and unexpected speed penalties

Pages per minute

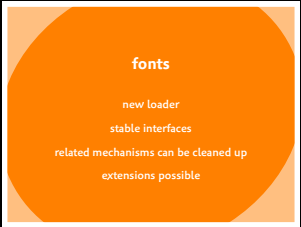
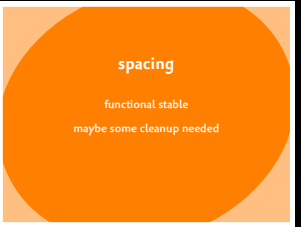
- we set up the baseline performance (in pages per second) as much as possible
- we try to identify and optimize critical routines, both at the `TeX` and `LaTeX` end
- we compare the used hardware machine and version of `LaTeX` and `CiTeX` matter

Microtime (10000 'count' 'pages')

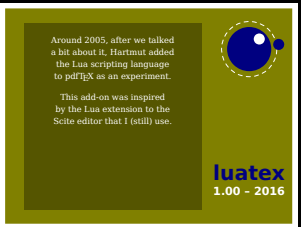
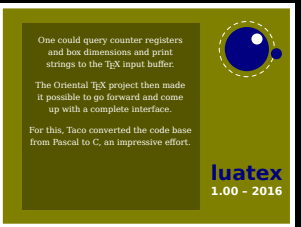
Page	Jan	Apr	May	May	init
1	2	2	2	4	4
10	15	17	17	36	37
20	50	109	110	217	236
100	185	234	239	509	512
1000	2315	298	298	648	657

2/98/2013, Lua^{TeX} 0.722, Dm6 6700, SSD, 16GB, 2.80 GHz 3840QMU, Windows 8 /32 bit
 06/2013, Lua^{TeX} 0.722, Dm6 6700, SSD, 16GB, 2.80 GHz 3840QMU, Windows 8 /64 bit

context-2015-status

	 fonts new loader stable interfaces related mechanisms can be cleaned up extensions possible	 hyphenation experimental normalization considered	 spacing functional stable maybe some cleanup needed
--	--	---	--

context-2016-luatex

 LuaTeX Version 1.00 ConTeXt meeting – September 2016	 After ten years of stepwise development and experimenting we release version 1.00 of LuaTeX during the 10th ConTeXt meeting in the Netherlands, September 2016. The interface is now rather stable and will not change significantly which means that one can write stable packages. So, it's time for a bit reflection as well as time to tell what we will be doing next. luatex 1.00 - 2016	 Around 2005, after we talked a bit about it, Hartmut added the Lua scripting language to pdfTeX as an experiment. This add-on was inspired by the Lua extension to the Scite editor that I (still) use. luatex 1.00 - 2016	 One could query counter registers and box dimensions and print strings to the TeX input buffer. The Oriental TeX project then made it possible to go forward and come up with a complete interface. For this, Taco converted the code base from Pascal to C, an impressive effort. luatex 1.00 - 2016
---	--	---	--

context-2017-css-selectors

CSS selectors	
Needed or not?	Simple formulas
Supported methods	Special cases
Filtering classes	Combinations
Filtering attributes	Compositors (1)
Combining methods	Compositors (2)
Child selectors	

- used in HTML styling
- a bit different from the path based method
- shortcuts for filtering by attribute
- class filtering is special because it checks for list
- one can select more at the same time
- performance is okay compared to path lookup
- selectors go between curly braces

```
$(html [#]) {foo bar .whatever, bar foo .whatever}}
```

The following methods are supported:	
<code>element</code>	all tags element
<code>element-1 > element-2</code>	all tags element-2 with parent tag element-1
<code>element-1 & element-2</code>	all tags element-2 preceded by tag element-1
<code>element-1 < element-2</code>	all tags element-2 preceded by tag element-1
<code>element-1 & element-2</code>	all tags element-2 inside tag element-1
<code>attribute</code>	all attributes
<code>[attribute=value]</code>	attribute equals value
<code>[attribute=value ...]</code>	attribute contains value (space is separator)
<code>[attribute*=value]</code>	attribute starts with value
<code>[attribute^*=value]</code>	attribute ends with value
<code>[attribute*~*=value]</code>	attribute contains value
<code>id</code>	id
<code>#id</code>	has class
<code>id=id</code>	has class
<code>id=id class=class</code>	the child of node <i>id</i>
<code>id=id class=class</code>	the child of node <i>id</i> from the end
<code>id=id class=class</code>	the first child
<code>id=id class=class</code>	the last child
<code>id=id class=class</code>	the match at index <i>n</i>
<code>id=id class=class</code>	the match at index <i>n</i> from the end
<code>id=id class=class</code>	the first match
<code>id=id class=class</code>	the last match
<code>id=id class=class</code>	only the match or nothing
<code>id=id class=class</code>	only the child or nothing
<code>empty</code>	empty
<code>!empty</code>	the whole tree

[illegible]

context-2017-features-chaintest



context-2017-features-kerntest



context-2017-features-pairtest

A B C D E F . . .

context-2017-features-singletest

A B C D E F . . .

context-2017-features-spacetest

A B C D E F G H . . .

context-2017-features-substitutiontest

abracadabra
XPrXcXdXPrX
YQrYcYdYQrY
XYPQrXYcXYdXYPQrXY
1raca2bra
aBraCadaBra
effe 1b3def !f . . .

context-2017-features

Font features

What are they
Substitution
Positioning
Ligatures
Kerning
Examples

CONT_EXt 2017 Mailbach

- built in substitution that is often optional like ligatures but for some languages mandatory
- built in positioning that is assumed to be applied like kerning, mark anchoring, kerning
- external properties like coloring, spacing, fallback combinations
- engine related tricks like expansion and protection
- tracing options
- whatever you like ... so let me know
- so in ConT_EXt we have font features (driven by font) and pseudo features (driven by additional needs)

What are they

- single: replace one by another
- alternate: replace one by one of a set
- multiple: replace one by multiple others
- ligature: replace multiple by one shape
- contextual lookups and replacements with look back and look ahead

Substitution

- single: repositioning a glyph (with optional marks), this includes traditional kerning
- pairwise: repositioning two adjacent glyphs (with optional marks)
- anchoring: often used for marks to base glyphs, ligatures and other marks
- attachment: often used for various scripts, putting glyphs in a word together
- contextual lookups and positioning with look back and look ahead

Positioning

context-2017-performance

Performance

Why bother
Possible bottlenecks
Measurements
Examples

CONT_EXt 2017 Mailbach

- because we don't want to waste time waiting
- because I get tired of ignorance related complaints
- because it (sometimes) can be a nice puzzle to improve performance
- because it occasionally reviewing makes code better

Why bother

- starting up
- loading fonts
- processing features
- applying trickery
- enabling tracing
- fonts
- L^AT_EX
- images

Possible bottlenecks

- how long does a run take
- how does the number of pages matter
- how many runs are needed
- start-up time
- processing pages
- finishing the document
- the console used

Measurements

context-2017-synctex

SYNCTEX

What is SYNCTEX
Disclaimer
What we do
How it works
What the user gets

CONT_EXt 2017 Mailbach

- it is a mechanism for going back from viewer to editor
- it uses an extra (cached) output file
- it adds an overhead of 2 to 15 percent runtime
- it is designed with a specific macro package in mind
- the rather generic approach works okay for simple document layouts
- but it often fails for projects that use multiple files
- and that moves information around like X_YL, recorded files

What is SYNCTEX

- all recently it was supported in ConT_EXt as it was
- there were no complaints, so it must have worked ok for most users
- we never used it ourselves because of mentioned reasons
- we only wanted to support it when it works ok in projects
- (think of thousands of X_YL with deeply nested inclusions in one document)
- but what we support now is purely based on personal experience
- we don't use it ourselves so feedback is welcome

Disclaimer

- the internal SYNCTEX mechanism is disabled
- when told so, ConT_EXt will look to its own code
- this is done by using L^AT_EX code to set the right information
- only source files that make sense are dealt with
- this protects the styles from unwanted changes
- within reasonable bounds X_YL is supported
- this also includes nested documents

What we do

context-2017-tables

The slide features a light blue background with a dark blue border. The title 'Tables' is in a large, bold, black serif font. Below it is a bulleted list of table types: 'Variants', 'Table', 'Tableset', 'Nested tables', 'Extreme tables', 'Long tables', and 'Frame tables'. The footer contains the text 'CONTeXt 2017 Maihach' in a black serif font.

- Good old TeX_ε, a wrapper about TeX's alignment.
- Running test, break across pages tables.
- Tables that behave like HTML tables therefore called natural tables.
- A variant on this that is more easy to extend, tagged tables.
- A low profile (invisible) mechanism that can open pages and breaks well.
- A way to make huge tables without overflowing TeX too soon.

- Based on the `TeXLive` macro package.
- Detailed control over spacing.
- Somewhat inconsistent spacing out of the box.
- Extended with extra features.
- Mostly rewritten but within the original concept.

- Mostly meant for tables that are part of the text flow.
- Breaks paragraphs across pages.
- Sort of compatible in control with $\mathcal{L}_A\mathcal{L}_B$.
- The system that I used most often.
- It uses multiple passes if needed.

Tabulate

context-2019-lmtx

Lean and mean
 LuaMetaTeX
 Hans & Alan
 ConTeXt meeting, September 2019

- **interference:** Co₂Q10, plain TGs and TG2s all have different demands (the source time to way from complex and more or not and much or less)
- **complexity:** the source time to way from complex and in the build (two only need LvsTG2s)
- **distributions:** to rule out generation stability for Co₂Q10 is a more popular than TGs (the above)
- **analogies:** experimental claims leads to vague out (LvsTG2s) and that triggers complex
- **metaphors:** mixing and side by side to stress "step by step" and "match instantly" wastes energy
- **arguments:** I got rid of "you need to stress the stability"
- **saying:** like "the material" = "becoming two times, so last keep experiments within the Co₂Q10 bubble"

LuaMetsTGX

How it became

- **duplication:** we don't need all that is currently in the Lua/Tex engine as we don't use it
- **source:** there is much less of it and we can get rid of web artifacts
- **compilation:** there was much more going on than was needed and we have those details
- **consistency:** in guarantee consistency with ConTeXt (the source code will be part of the source distribution (once it is finished))
- **marketing:** this way the relation with ConTeXt and its users is more clear
- **platforms:** we can move forward and experiment without the danger of running into problems with non supported platforms, "we did it your own way"
- **platforms:** playing a bit more with the bits and pieces that are responsible for most (interesting) issues, like the (superscript) page header

- binary:** there is only one relatively small binary module (that does all things needed)
- code base:** there remains an extra source tree, but it's a small convenience to around 2-3k
- user controls:** if needed users can compile the program on we let it download
- future code:** we can make future forward and improve
- modules:** a code base with the latest LuaTgX, mhtml and Lua
- side effects:** we drop LuaJIT as it doesn't work up (and benefits are too small)
- design:** we bring Lua together as it separates the keep the code small and outside the program
- independent:** we keep all our dependencies on external, however, we keep all we need in the code base (we only use a few small third party libraries)

context-2020-concepts

CONCEPTS

experiments turned features

context 2020 meeting

Experiments

There have been quite some experiments. Some results were rejected, some kept. There are a few (that come to mind). This talk is a mix of summary, discussion and some demos.

concepts – model 2020 meeting – experiments

Math

There are a couple of additional features in the math engine. Most concerns a bit more control over nested behavior, but none are used of now:

```
test $a = b { discretionary class 2 $[4] $[5] $[6] } <5 test
```

When there is enough room this will give

```
test a = b a c test
```

When `!infix` is limited we get:

```
test
a =
b <
c
d
e
f
```

categories — created 2020 meeting — math

[illegible]

LUAMETATEX
where do we stand
context 2020 meeting

baamatax - context 2020 meeting - when it started

luametaux - context 2020 meeting - why it started

huametatex - content 2020 meeting - the development

MKII MKIV LMTX
 where does it end
 context 2020 meeting

METAFUN
simple fonts
context 2020 meeting

ConT_FXt Meeting

context-2020-svg

SVG GRAPHICS

some demos and discussion

context 2020 meeting

Wrapup

- It is required to be Scalable Vector Graphics.
- It is an example of application and format standard.
- It started out simple, kind of expanded PostScript in text format.
- It took a while to be picked up as output format.
- In practice you get the same money build up as in other vector formats.
- This is a side effect of often unstructured editing.

svg graphics -- context 2020 meeting -- wrapup

Properties

- Properties can be set on attributes or on elements (key/values).
- Properties can be set in the style attribute (sometimes separated key/values).
- Properties can be set via one or more class assignments.
- Properties can be bound to a specific element.
- Properties can be inherited from an ancestor (sometimes request).
- Properties can be inherited (inherited), overriden (parent, style), editors can add their own, etc. ... it's kind of a mess.

svg graphics -- context 2020 meeting -- properties

Side effects

```
usemmla[gnuplot]

vectorial figure
[context-2020-gnome.gn]
[conversion-svg, width=4cm,
background=white, backgroundcolor=white]

vectorial figure
[context-2020-gnome.gn]
[conversion-svg, width=4cm,
background=white, backgroundcolor=white]

width
[width=4cm]
[if named]
[background=white, backgroundcolor=white]
[includegnuplotfile[context-2020-tin.svg]]

svg graphics -- context 2020 meeting -- side effects
```

context-2020-tokens

TOKENS

tokens as I see them

context 2020 meeting

About tokens

- Like indices, it's a common term used in programming.
- In TeX The Program tokens and indices are therefore almost generic.
- For most users they are irrelevant concepts.
- But we will explain them anyway.
- Let's try to avoid the misinterpretation which sometimes based on the community.
- So ... (over) correct you as long as you don't correct me.
- Let's now enter the world of tokens in the naive way.

tokens -- context 2020 meeting -- about tokens

What are tokens

- It is an internal data structure, effectively a 32 bit integer.
- This integer encodes a (sometimes) (optional) and an (other code) (optional).
- But often it's not a character but more a sub command.
- Token are either expanded (internally) or stored.
- When they are stored they are part of a larger data structure, a memory word.
- Token memory is an array of such memory words.
- The token memory 'word' has two integers, a token value and an index into token memory.
- The way TeX can have forward linked lists of tokens.
- A hash table maps external sequences into indices into token memory.

tokens -- context 2020 meeting -- what are tokens

Some implementation details

- Sometimes there is special hard token at the start.
- A hard token makes for easier appending of extra tokens.
- Should last use the hard code for a reference count.
- Original TeX uses global temporary lists.
- These are used when we expand tokens and need to report tokens.
- This is not needed when we just tokenize (which we do a lot in LuaTeX).
- So this is all optimized for performance and memory consumption.
- Fixed tokens are collected in a cache so tokens can get scattered.
- In LuaTeX we try as close to original TeX as possible.
- But the last (mis)feature seems to be not actually down.

tokens -- context 2020 meeting -- some implementation details

context-2021-compactfonts

COMPACT FONTS

a different approach to scaling

context 2021 meeting

How TeX uses fonts

- In a TeX engine there is a 2-to-1 mapping from characters in the input to what is in a font.
- Combinations of characters can be mapped into one shape (ligatures).
- Implementations characterized by the input mapping of what which sequence come (ligatures).
- In the typical result characters (glyphs) can be implemented relatively naive.
- This all happens in a very interesting way (e.g. inside the pdf builder).

compact fonts -- context 2021 meeting -- how TeX uses fonts

How traditional TeX sees fonts

- A font is just a (binary) blob of data with information where characters have a width, height, depth and some corrections.
- The engine only needs the dimensions and when the same font is used with a different scale a copy with scaled dimensions is used.
- There are some exceptions for ligatures and (some) complex or related such as (some) shapes (like arrows, wide arrows, boxes and radicals).
- There are some specifications for kerning.
- A handful of font parameters is provided in order to control for better spacing.
- Virtual fonts are just named fonts but they have support for the backward to forward shapes. TeX uses the normal metric file, the backward also the virtual specification file.

compact fonts -- context 2021 meeting -- how traditional TeX sees fonts

Unicode engines

- There are no fundamental differences between a traditional engine and an Unicode aware one with respect to what TeX needs from fonts: dimensions.
- There can be more than 256 characters in a font.
- Some mechanisms have to be present for applying font features like ligatures and kerning.
- In LuaTeX implementation, ligatures and kerning are clearly separate steps.
- In LuaTeX collects the list of things with fonts and characters.
- Hash fonts are handled differently and there is some info that comes from the font.
- In LuaTeX features like expansion are implemented differently. In no copies of fonts are made and applied expansion works with the glyphs.

compact fonts -- context 2021 meeting -- unicode engines

PROGRAMMING
local control
context 2021 meeting

- Can't be in several call reading from reading module:
 - there must only be reading from the source files. Characters are picked up and interpreted. Depending on what takes it becomes some action like plus
- 1 `\\various\\dots to \\input{2}` `\\various{}` the count `\\multicolumn{count}{by 4}`
- The 2 gets triggered because characters like the ones on text
- The `\\various{}` primitive triggers picking up a regular number, then goes on scanning for a bus (separator) and that itself will trigger a sequence if whatever until the group ends
- The count primitive triggers scanning for a regular number (or reference) and then scans for a number: the output sign is optional.
- The primitive injects some value into the current input stream (if does so) by entering a new input line
- `\\input{2}` primitive picks up a regular sequence and multiplies that by the next scanned number. The by is optional.
- Printing from LaTeX and scanning (things with e.g. `\\various{}` in LaTeX reading (pseudo) files.

programming – context 2021 meeting – expansion

[illegible][illegible]

LUAMETAFUN
the new interfaces
context 2021 meeting

- The core of the language is still `JSX`, it all starts there.
- The `JSX` integers are opened up by `Lua`. We can call out to `Lua` from `JSX` and to some extent from `Lua` to `JSX`. Quite often we push something through the input channels.
- The `MetaScript` library is accessed from `Lua`. So from within `JSX` there is always `Lua` in between. Usually go back to `Lua`. The library can also call out to `Lua` and from there to `JSX`.
- This means that if we throw many components of `LuaMetaScript` can talk to each other and use each others capabilities
- With `Lua` in the center, we also have access to other mechanisms, for instance fonts, graphics and libraries.
- In `ConScript` the `Lua` language also permits using `xml`, `json`, `csv`, `sql` databases and other input that can be dealt with programmatically.
- All has been reasonably optimized for efficiency and performance.

- Turning MetaPost into a library has been a subject of the *LaTeX* project. The semi-official team (Trevi, Jacko, Hans, Knigge) got John Hildreth blessing.
- This was a rather massive (and expensive) operation by Trevis because many more modules had to be reworked to make sure that such different backends were possible. All with remaining perfect DVI compatibility.
- The MetaPost library serves both the stand alone program and *LaTeX*.
- This means the PostScript backend is built in plus some basic (Type1) font handling. We support *pdf-output* via the MetaPost *LaTeX* backend (in Meta) that is done by parsing the PostScript and specials.
- In addition there is *ps* and *eps* output. It helps that MetaPost output is rather simple.
- The *LaTeX* engine uses the *LaTeX* backend which represents the result in *LaTeX* tables resembling the *LaTeX* internal representations.
- The library supports scaled and double (internal) but also binary and decimal number modules that you linked to libraries.

- We don't need the Postscript package (which only does Typeset anyway).
- We also have no use for `wg` and `pgk` output.
- The binary number model has no advantages over the decimal one but brings quite some dependency with a library code.
- The Typeset flag support is not used in Coq2CvC because we handle text differently.
- All this means that we can do with a smaller (simplified) MetaPost library.
- The codebase has been overhauled. We still have... a few (few!) but we use a `Less` script to convert that to C which means we have better control over how it comes out.
- As with `LaTeX` the file is, handling metafile etc. now largely goes via `Latex`. It is more integrated.
- The same is true for examining invariants and return values (exceptions). This also made for more uniformity.
- Memory management (allocation) is under engine control (with `TS` and `Loc`) we use a common high performance allocation library.

MATH
complete control
context 2021 meeting

- Traditional TeX has set the benchmark for typesetting math.
- It uses three categories of fonts: *alphanumeric*, *symbol*, *calligraphic*.
- Limitations in fonts have led to some curious handling of dimensions.
- The fact that there is *never* macros for multispaces *handing and processing*.
- The last family counts (when entering the second pass) so one has to use very controlled font switching.
- Some features (limits, integrals, primes) rely on special macros and parsing.
- One can end-up in tricky font switching. Also, quite some fonts are loaded in order to set up the machinery.

- We now use OpenType math fonts. In fact, in TeX_{IV} we always used only one font instance (unless we mix font families).
- We operate in the Unicode domain, so in the end there was no need to build the number of families in TeX_{IV}.
- In ConTeXt we use only a few font families: regular, x2, bold, bold x2.
- All the tricky stuff is done with the help of Lua. This already started very early in the LaTeX project and has not changed.
- But we still run into issues because the available fonts are inconsistent, incompatible, have issues and that is unlikely to change.
- We have a virtual font system in place that was used during the transition (even so fonts were available).
- In ConTeXt there are various ways to deal with shortcomings in or extensions to fonts.
- At some point we need to make up our minds, accept the issues with fonts, and just fix things runtime.

- In `Lua5.2`, we started with a hybrid approach but eventually ended up with more split code paths.
- In `LuaMetaMIX`, nearly all aspects of the engine had made configurable and are under user control.
- This permits experiments where we can apply old methods onto new hosts.

context-2021-overloadprotection

OVERLOAD PROTECTION

the downside of macros

context 2021 meeting

Primitives

A TeX engine comes with a whole set of primitive operations for:

- accessing internal variables
- defining macros
- accessing expansion
- constructing boxes
- building pages
- defining characters (text and math)
- inserting boxes, glue and penalties
- defining fonts
- dealing with language implementations
- listing properties
- manipulating tables
- managing macros
- handling marks
- grouping
- subexpressions
- tracing

overload protection — context 2021 meeting — primitives

Macros

- Macros are commands defined by the user under a macro package.
- They can overload a primitive which then can confuse the whole machinery.
- A macro package can also primitives for instance `\relax` can be replaced by `\font\relax after` `\let\let\after` before.
- The only when at definition time the `_` as a letter. By using such a character some primitive against overload is provided.
- In ConTeXt we often, usually abuse like `\internalize` but of course these can also be overloaded.
- We only overload a very few primitives, for instance `\language`.
- Users who overload primitives are 'on their own' and 'without support'.
- An easy way to protect yourself is using `\ContextCase` macros.

overload protection — context 2021 meeting — macros

Overload protection

- The LuaTMTeX engine has overload protection built in for the TeX engine as well as provides means to do that for MetaPost.
- In L^AT_EX all commands have been tagged accordingly (which was quite some work).
- Processing a system macro will then do no harm.
- Overload protection is off by default but can be turned on:

```
\setlocalobjectives{\overloadno\overloadno\overloadno}
```
- LuaTM always goes with the error message and make sure that the macros obey the rules.
- Modules and/or styles (and in a few cases the more could can cheat and use `\unprotectedload`):

```
\unprotectedload  
.....  
\unprotectedload
```

overload protection — context 2021 meeting — overload protection

context-2021-paragraphs

PARAGRAPHS

a bit of an upgrade

context 2021 meeting

Note

- Some of the following already is present for a while and has been discussed at previous meet-ups.
- But... occasionally some minor tweaks you added as consider this to be an update.

paragraphs — context 2021 meeting — note

Spacing

- Spaces in TeX become glue nodes (with optional stretch and shrink).
- In traditional TeX these glue nodes are not extended copies of the current spacing related values.
- In LuaTMTeX we make real copies so that when we mess with the node list changes to glue don't affect other instances.

paragraphs — context 2021 meeting — spacing

Parameters

- In traditional TeX the paragraphs based properties that are in effect when (text happens are used when breaking sections).
- In LuaTMTeX the paragraphs based properties are stored with the paragraph and can be frozen when they are set.
- This gives a more predictable (and robust) way of manipulating a paragraph.
- We can for instance get rid of grouping side effects that interfere with `\overprint`.
- Currently three dozen parameters are tracked but they are grouped in categories.
(show code and examples)

paragraphs — context 2021 meeting — parameters

context-2021-programming

PROGRAMMING

the way ConTeXt is set up

context 2021 meeting

Levels

When you look at ConTeXt bottom-up (engine → interface) you will notice:

1. **primitives**: that is what the engine comes with
2. **infrastructure**: basic management of data structures
3. **helpers**: macros that hide complexity
4. **subsystems**: reflections of macros that implement functionality
5. **mechanisms**: these combine various subsystems
6. **modules**: extra functionality (used 1-5)
7. **styles**: handling sources and layout issues 4-6)

Users normally use ConTeXt top-down (style → layout → backing).

programming — context 2021 meeting — levels

Styles

- These are probably solutions for common as well as rare situations.
- The system comes with some styles: the `x-y` files.
- Right from the start the idea was that you get some reasonable default.
- And if you want more you yourself define your style as you go.
- It really is a part of the game: exploration.
- Solving your problem is a nice challenge.
- If you want a completely predefined setup, drop somewhere else.

programming — context 2021 meeting — styles

Modules

- A lot of functionality is built in.
- This helps to keep the system consistent.
- We could cheat and ship thousands of low-level styles but don't do that.
- There are a few mechanisms that don't really fit into the core, so these are implemented as modules that do fit into the interface. See `u` and `v` files.
- Users can build and share their solutions which has resulted in some third party modules: the `x-y` files.
- For `u` is low, often still provide files `u` or `v` name schemes.

programming — context 2021 meeting — modules

TUG 2001

A *Typt* Odyssey

- What way are we heading
- Will there be documents
- Is typography still needed
- Are we still talking *Typt*

Used as it, the main source of information in books. In the next couple of slides, I will present some quotes from books I read the last couple of years, written by: Arthur Clarke, Greg Bear, Graham Hancock, Peter Wilton and Michael Shuster, Jared Diamond, Richard Taft, Peter Ward and Donald Brown, Steve Ranch and Seryl Kiser, Richard Kadey, Susan Satterworth and of course Donald Knuth.

[Arthur: Cries] Although I've never dismembered 2001 as a stock prediction—but as none of a million, a way things could shake—I have long kept track, informally, of how our nation compares with computer science reality. Some things we got right—were righter than we ever had reason to suspect. Crime, yes, you could have known. Monetary policy, the use of the atomic predicted in 1945 is okay, but with respect to computers a couple of points are missed they have become crucial. All is far from operations, national speech, security and spending are not really visible, still, tolerance is there, we have left, graphical user interfaces and windows, don't communicate in textual messages, have mice and other means of input.

After a short walk through a tunnel packed with paper and cables, and echoing hollowly with rhythmic thumping and thumpings, they arrived in executive territory, and Floyd found himself back in the familiar environment of typewriters, office computers, girl assistants, wall charts and ringing telephones.

hans jagten

**zapfino, a
torture test
for luatex**

ing. wend van der gijp

for luatex

loading fonts

the **spentype** font reader is borrowed from **fontforge**

once it was ready, we could look into such a font

it tooks while to figure out the format due to rather fuzzy specs

it took us even more time to find out that the loader was flawed

one reason was that fonts themselves may have bugs or be incomplete

then we changed to **fontforge** version 2

this made the missing pieces surface in more complex feature handling

while implementing features the new table format was cleaned up

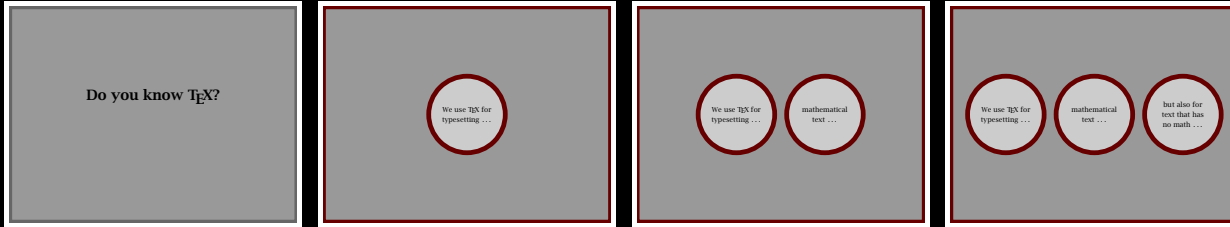
making it work

- the **fonttools** library loads the fonts into memory
- the internal data structures are mapped into a **font table**
- we enhance this table a bit (extra hashes as well as rearranging glyphs)
- we save the table in a cache and compile it to bytecode
- after that reloading is fast and efficient
- when defining a font instance we share data as much as possible
- the open type specific table is translated into a **ttf** one
- but for handling features we keep the original table available

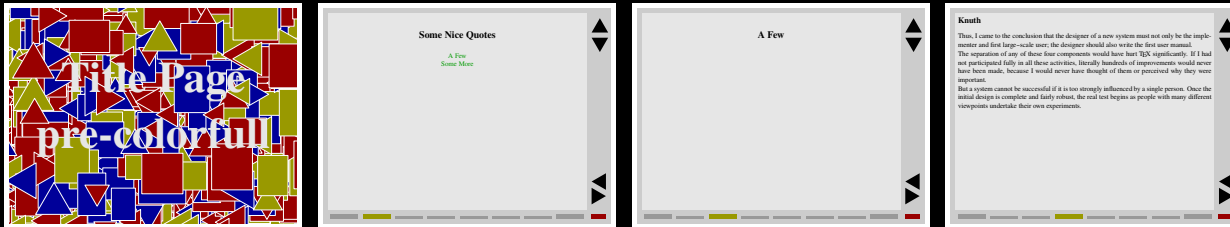
user control

- in context *mkiv* currently we provide several interfaces
- the *xates* syntax is (more or less) supported
- there is a higher level interface to associating features with fonts
- there will be a font instance independent feature switch mechanism
- there can be generic treatments unrelated to the specific font
- fonts can be accessed by various names
- but ... using features demands that users know what they're doing

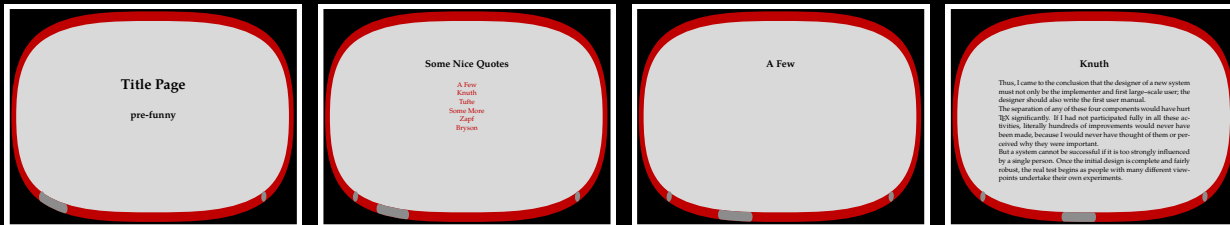
present-balls-001



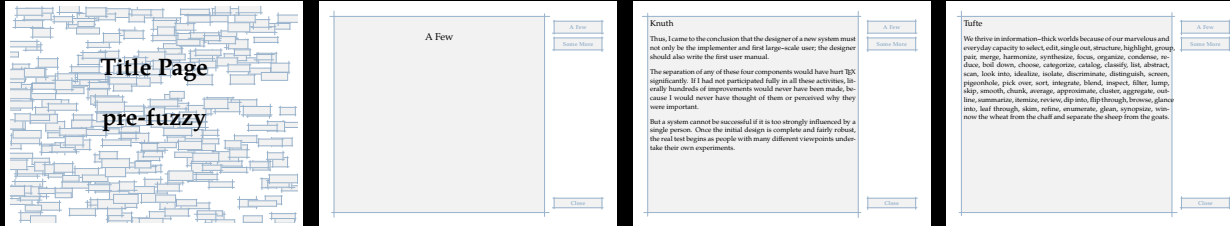
present-colorful-001



present-funny-001



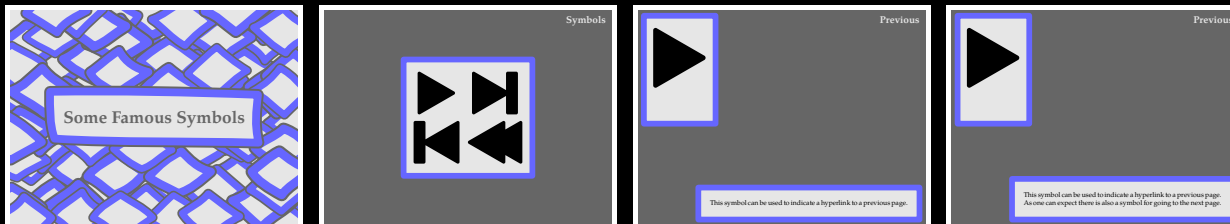
present-fuzzy-001



present-green-001

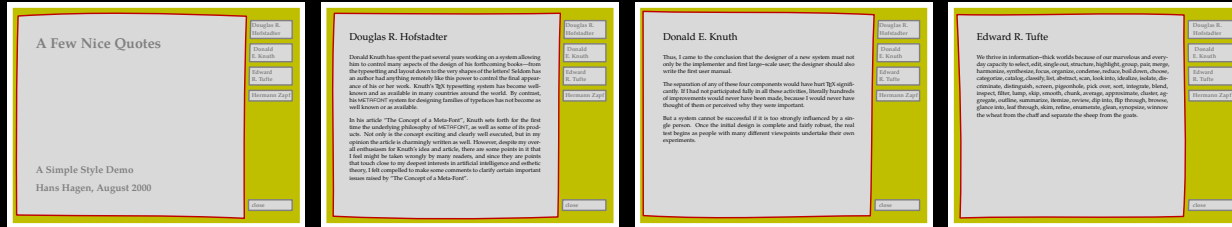


present-grow-001

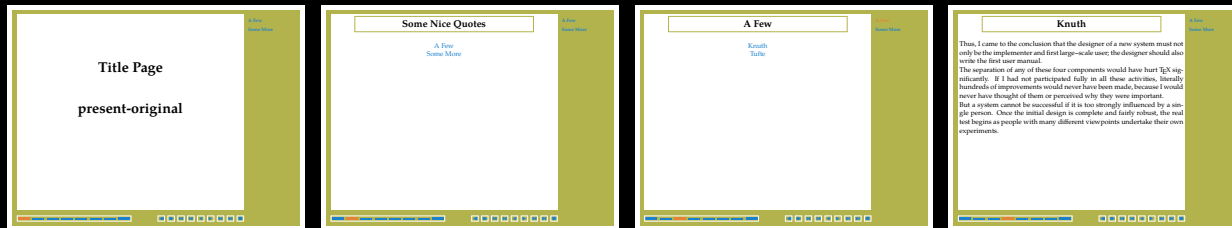


Examples

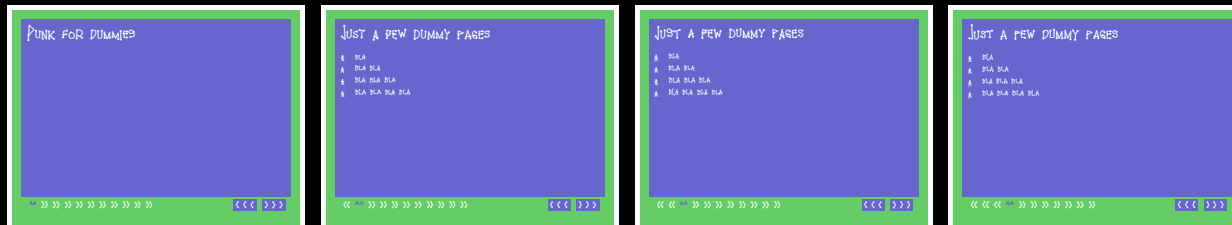
present-organic-001



present-original-001

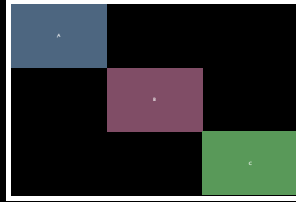
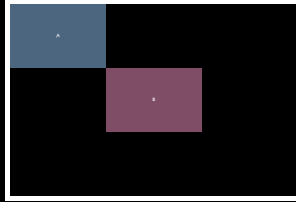


present-punk-001

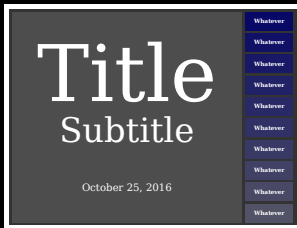


Examples

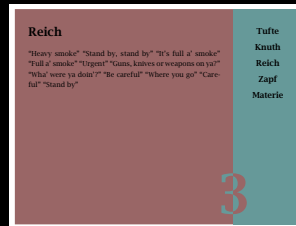
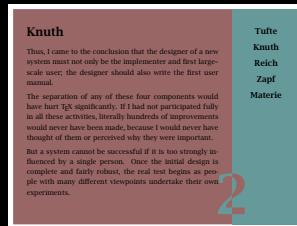
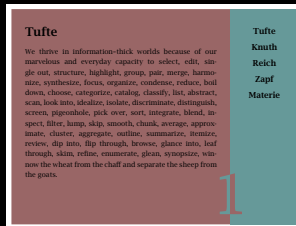
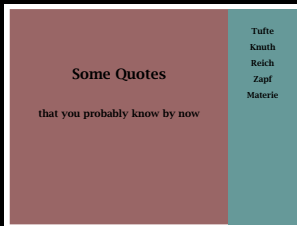
present-random-001



present-shaded-001



present-split-001



present-stepper-001

Stepwise
Refinement

Topics

Female Artists
Male Composers

And Some More

Female Artists

- Fiona Apple
- Tori Amos
- Kate Bush
- Heather Nova
- Alanis Morissette
- Suzanne Vega

Male Composers

- John Adams
- Steve Reich
- Louis Andriessen
- Olivier Messiaen

present-steps-001

Contents

1 Step Set 1 2
2 Step Set 2 1

1 Step Set 1

1 Step Set 1

STEP ONE

1 Step Set 1

STEP ONE

STEP TWO

present-tiles-001

Whatever We
Want Here

Whatever We
Want There

Topic 1	Topic 2	Topic 3
Topic 4	Topic 5	Topic 6
Topic 7	Topic 8	Topic 9
Topic 10	Topic 11	Topic 12

Topic 1

We thrive in information-thick worlds because of our marvelous and everyday capacity to select, edit, single out, structure, highlight, group, put energy, harmonize, synthesize, focus, organize, condense, reduce, boil down, choose, categorize, catalog, classify, list, abstract, scan, look into, identify, isolate, discriminate, distinguish, screen, pigeonhole, pick-over, sort, integrate, blend, inspect, filter, keep, skip, smooth, check, average, approximate, cluster, aggregate, outline, summarize, itemize, review, dip into, flip through, browse, glance into, lead through, skim, refine, enumerate, plan, reorganize, sequence the subject from the chaff and separate the sheep from the goats.

Buttons (roll-over in aacrobath):

click left top home
click right top contents
click left bottom previous
click left bottom next page

Topic 2

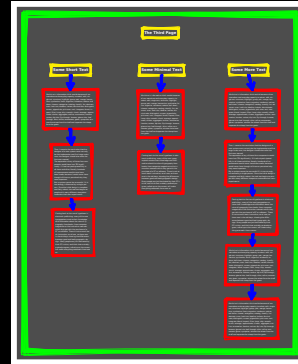
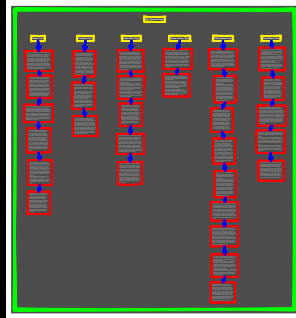
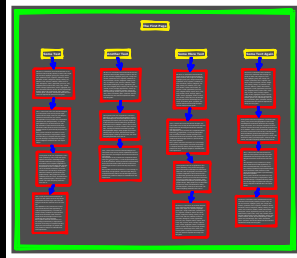
We thrive in information-thick worlds because of our marvelous and everyday capacity to select, edit, single out, structure, highlight, group, put energy, harmonize, synthesize, focus, organize, condense, reduce, boil down, choose, categorize, catalog, classify, list, abstract, scan, look into, identify, isolate, discriminate, distinguish, screen, pigeonhole, pick-over, sort, integrate, blend, inspect, filter, keep, skip, smooth, check, average, approximate, cluster, aggregate, outline, summarize, itemize, review, dip into, flip through, browse, glance into, lead through, skim, refine, enumerate, plan, reorganize, sequence the subject from the chaff and separate the sheep from the goats.

Buttons (roll-over in aacrobath):

click left top home
click right top contents
click left bottom previous
click left bottom next page

Examples

**About Text
Today's Talk
Hans Hagen**



Quotes, Quotes
and more quotes

sample. Then, I came to the conclusion that the designer of a new system must not only rely on the implementer and first large-scale user, the designer should also write the first user manual. The first user manual is not a luxury, it is a necessity. I did not participate fully in all these activities, but the whole cycle of implementation would have been made less chaotic if I would have thought of them or pointed out why they were important.

But a system cannot be successful if it is not strongly influenced by a single person. Once the initial design is complete and fairly robust, the rest belongs to people with many different viewpoints outside their own organizations.

Because of social mechanisms, too, structures, habits, ideas, opinions, conditions like time, class, sex, abstract, detailed, concrete, project, effort, time, space, money, power, status, prestige, reputation, self-interest, sympathy, between the person and the system, between the person and the group.

Tuftse

[illegible][illegible]